

Synopticon: Consensus-Based Cheating Detection System for Competitive Games

JEUK KANG, School of Cybersecurity, Korea University, Korea

JUNGHEUM PARK, School of Cybersecurity, Korea University, Korea

Cheating in online games has serious impacts on the game industry in various ways. Although extensive research has been conducted in this field, most studies have focused on a specific genre, the Massively Multiplayer Online Role-Playing Game (MMORPG). On the other hand, competitive genres—such as Multiplayer Online Battle Arena (MOBA), First Person Shooter (FPS), Real Time Strategy (RTS), Action—have received less attention compared to MMORPG, as both, the difficulty in detecting cheating users and the demand for sophisticated techniques and data, make them less attractive. Rather than take the harder path, game companies, which make competitive games, have adopted an anti-cheat solution and have urged users to install it before playing their games. However, the anti-cheat solution demands kernel-level permission, and its operations are concealed from users. Due to its features, it has been used for attacks by hackers taking advantage of its high authority, and users are posing questions about their privacy. In this paper, we introduce SYNOPTICON, a novel cheating detection system. SYNOPTICON detects cheating users by using normal users' consensus, which is achieved by users' votes. SYNOPTICON consists of a detection method on client-side and a voting system on server-side. When abnormal behavior of cheating users is detected by a client, it casts a vote to the server, which then distinguishes between cheating users and normal users based on the aggregated votes and the consensus achieved by the votes. This design allows SYNOPTICON to address the key challenges faced by cheating detection systems. SYNOPTICON has been tested in both a simulation and a real-world FPS game. With the simulation, we verify the capabilities and requirements of SYNOPTICON, and, with the real-world game, we evaluate its effectiveness in identifying cheating users. As a result, SYNOPTICON successfully detected cheating users. In addition, we demonstrate its applicability and sustainability in managing online games through a real-like operational environment developed using public datasets.

Additional Key Words and Phrases: Game Security, Online Game, Cheating Detection, Consensus Algorithm

ACM Reference Format:

Jeuk Kang and Jungheum Park. 2026. Synopticon: Consensus-Based Cheating Detection System for Competitive Games. 1, 1 (May 2026), 22 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Cheating in computer games is often perceived as acts of *gaining advantages* by abusing a game system ([8]). In the early stage of the game industry, cheating emerged from single-playing games and was used for improving a player's personal satisfaction by manipulating the difficulty or improving the handiness of a game ([35]). As time passed, online multi-playing games appeared, and soon, became the mainstream of the game industry. Cheating was also on the stream, and its characteristics have changed reflecting the changes in the industry. Most recently, cheating is interfering in interactions between online players (or users) and is damaging the game industry ([5]).

Authors' Contact Information: Jeuk Kang, gangjeuk@korea.ac.kr, School of Cybersecurity, Korea University, Seoul, Korea; Jungheum Park, jungheumpark@korea.ac.kr, School of Cybersecurity, Korea University, Seoul, Korea.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

To solve these, numerous studies on anti-cheat methods, which detect and counter cheating, have been conducted, yet the majority of these studies have concentrated on one specific genre of online game named Massively Multiplayer Online Role-Playing Game (MMORPG). In addition, much of the research has targeted cheating aimed at financial gain. Survey papers ([17, 21, 49]), that went over anti-cheat methods indicate that most research has focused on MMORPGs, while relatively few studies addressing competitive genres—such as Multiplayer Online Battle Arena (MOBA), First Person Shooter (FPS), Real Time Strategy (RTS), Action. This disparity is largely due to the fact that the *means* and *objectives* of cheating vary significantly depending on the *ultimate goal* of each genre. In a single-player game, the goal is to complete a game or simply to enjoy a game. In MMORPG, it is usually set to the growth of players’ characters, in line with the Role-Playing nature of the genre. In competitive genre, it is set to a victory in competition between players. Such differences in the *goals* make distinct differences to cheating as well. For example, in competitive genre, cheating operates as a supporter and guarantees a winning by propping up the player’s performance and by providing additional information that cannot be acquired in a normal way. Further discussion on the *means* and *objectives* of cheating across various game genres will be provided in [Section 2](#).

However, there are several challenges with the existing cheating detection methods that have been proposed for the competitive genre.

Ch1: Lack of consideration for liveness Our investigation in [Section 2](#) reveals that most of the anti-cheat methods applied in MMORPG rely on filtering out cheating users by analyzing long-term logs on game servers. However, such approaches are inadequate for competitive genre; mainly because, many competitive games are played on a match-basis. In these games, a matching system sets two groups (or teams, e.g., N vs N) of users, and matches between two groups are played. In contrast, users in MMORPG are playing a game in a virtual world with a continuous timeline. Therefore, cheating detection in competitive genre should be rapid and performed within a single match. In short, the method must satisfy the liveness, ensuring quick detection and high accuracy during a match. Moreover, the liveness is crucial for an aspect of user experiences; because, in competitive genre, cheating directly influences normal users and harms their impression on a game by spoiling user experiences and giving a sense of losing during the whole match time.

Ch2: High overhead The introduction of cheating detection systems inevitably incurs overhead on servers or clients, and many methods applied in MMORPG pass on their costs to servers. However, this approach is less feasible for competitive genre; because, it requires more sophisticated datasets and techniques, both increasing the cost. For instance, Valve, an American game company, implemented a cheating detection system on their servers to filter out cheating users in one of their games named ‘Counter-Strike: Global Offensive’. Consequently, they had to use the server machines with 3,456-cores for one day ([28]), and it is expected that Valve had to spend a few million dollars at least ([23]) to maintain the server. Some game companies tried to pass on their costs to users by involving users as judges in the ruling process ([37, 44, 45]). However, their attempt failed because of biases in human judgments and the inefficiency of the system itself ([37]).

Ch3: Low applicability Client-side methods, on the other hand, have suffered from their low applicability. Recent studies already reached the state-of-the-art in detection rate of cheating users by using a mix of sophisticated machine learning methods and detailed datasets; however, regardless of its accuracy, it is still remained vulnerable to tampering by clients. Several papers adopted the Trusted Execution Environment (TEE) ([7, 20, 34]) to prevent manipulations by clients. However, the use of TEE is yet impractical—due to the limited number of CPU models that support TEE in the personal computer market, and the inaccessibility of TEE for game companies in the mobile market—which make it hard for the companies to apply TEE on their games.

Ch4: Low sustainability Cheating detection systems need constant updates and re-adjustment, as parameters and variables in games, which researchers used for the optimization of their methods, can change throughout the updates on games, such as re-balancing patches. In addition, the systems also have to deal with new cheating methods. Otherwise, they will be outdated soon. Recent research for MMORPG has just started to address the importance of sustainability and has begun to discuss the operational aspects of their methods ([24, 43, 50]). In competitive genre, however, it hasn't been discussed yet.

Giant game companies (e.g., EA, Epic Games, Riot of Tencent, Activision Blizzard of Microsoft) handled these challenges with client-side anti-cheat solutions ([1, 12, 39, 46]). Many of them are forcing users to install it before playing their games. For instance, Epic Games, known for its famous game 'Fortnite' with 400 million global users ([4]), mandates the installation of their anti-cheat solution ([14]). These solutions run at kernel-level and operate by monitoring external programs' interference in a game-client, a program that runs a game. Unlike other methods of previous studies, they do not rely on sophisticated machine learning techniques and detailed datasets (**Ch3**). They operate on client-side (**Ch2**) and run in real-time (**Ch1**). Finally, they are continually updated and managed by fixing security loopholes and responding to evolving cheating methods (**Ch4**).

However, the cost of anti-cheat solutions, which may seem reasonable solutions for game companies, are paid by clients' privacy. Vulnerabilities in these ([29–31]) have been used by attackers ([41]), and the high privileges granted to the solutions—intended to assure their unconstrained operations—is then offering advantages to attackers ([11, 27]). Furthermore, because of its unconstrained operations, it has been accused of illegal gathering of information. Game companies are trying to refute these accusations against them. For instance, Riot Games—which developed its own anti-cheat solution named *Vanguard* ([40]) and integrated the solution with its games—has offered \$100,000 bug-bounty reward to demonstrate its commitment on security ([38]). Likewise, others have made efforts to prove their solutions are both safe and privacy-preserving ([1, 12, 33, 39]). Nevertheless, despite these efforts, anti-cheat solutions are still yet bypass-able ([2, 16]), and threats against the solutions are ongoing problems.

In this paper, we introduce SYNOPTICON, a novel cheating detection system based on the consensus of normal users. Inspired by Jeremy Bentham's concept of 'Panopticon' ([6]), SYNOPTICON leverage mutual surveillance, where users evaluate each other, and SYNOPTICON finds bad guys by using a consensus among users achieved by the mutual evaluations. By utilizing the collective judgment of users as the **Ground truth**, SYNOPTICON effectively differentiates between cheating users and normal users. SYNOPTICON can be implemented on client-side (**Ch2**) and overcomes the inherent limitations of client-side methods (**Ch3**)—without the need for additional technologies, such as TEE. We evaluate SYNOPTICON using both simulations and a real-world dataset. Our evaluation demonstrates that SYNOPTICON can detect cheating users with high accuracy and speed (**Ch1**). Furthermore, we explore how SYNOPTICON can be applied in terms of game service management and operating (**Ch4**).

Finally, a proof-of-concept version of SYNOPTICON and the associated datasets are [publicly available](#), allowing this work to be reproduced.

2 Background

2.1 Type of game cheating

As mentioned in [Section 1](#), cheatings are typically employed to gain *benefits*, that cannot be obtained in a normal way, and the nature of these benefits varies by the genre and *ultimate goal* of a game. If a goal is set for the growth of characters, cheating can be used to earn game money to purchase new gadgets (e.g., new swords and shields) and to

Table 1. Typical cheating behaviors in online multi-playing games

Genre	Subgenre	Cheating	Description
Common	-	Exploits	Using of bugs and manipulation of certain features on games (e.g., speed hack that allows rapid move, ghosting that is being invisible)
Role-Playing	MMORPG	Game bots	Operate automated bots and gather virtual property or achievement
Competitive	FPS	Scripting	Automate certain behavior or action and assure fast sequential inputs or rapid response time (e.g., aimbots on FPS)
	MOBA		
	RTS	Extra-Sensory Perception (ESP)	Disclose hidden information, such as, location of other users behind wall which should not be displayed in normal
	Action		

upgrade the ability of their character (e.g., power and speed). When it is a victory in competition ([25]), cheating can serve as an assistant by encouraging cheating users to perform beyond their innate abilities. We have classified types of game cheating according to game genres and cheating methods as listed on Table 1.

2.2 Existing cheating detection methods

Table 2. Number of existing research papers on MMORPG

Detection side	Used Method			
	STAT	ML	SIMI	NET
Server-side	10	21	3	8
Client-side	-	3	-	-
Network-side	4	-	-	-

STAT: Statistics analysis, ML: Machine learning
SIMI: Similarity analysis, NET: Network based

Cheating detection methods can be categorized into two primary types depending on where they are applied: client-side and server-side. We reviewed survey papers ([17, 21, 49]) that investigated cheating detection methods targeting MMORPG genre, which has been researched most. Detection methods in the papers were classified according to the type of the methods—whether client-side or server-side—as well as the specific techniques employed. As shown in Table 2, most research in MMORPG genre has adopted server-side, and its methods are mainly about analysis of server-side logs. This approach works well in detecting *gamebots*, as their behavior patterns differ significantly from those of normal users in a long-term view. However, it is less effective for detecting scripting cheating in competitive genre, where games are typically played on match-basis.

2.3 Research scope

Before explaining SYNOPTICON, we clarify the scope of its applicability. From the top of the Table 1, first, MMORPG and *gamebots* are outside the scope of this study. Regarding Role-Playing, extensive research has already been conducted on *gamebot* detection, and several state-of-the-art methods have been developed ([24, 32, 43, 50]). Second, our focus is on the competitive genre and the detection of scripting, where the advantages of SYNOPTICON can be used most effectively. Meanwhile, other cheatings on competitive genre are not addressed in the paper. In the case of *exploits*, it can easily be

defended in modern online games, as most of the states and features of games are calculated and managed on server-side, not client-side. Epic Games—which developed the *Unreal Engine*, one of two central pillars of 3D game engine—explains to developers that “cheats allowing players to fly, teleport, or move extremely fast can usually be completely prevented at the game architecture level by giving game servers authority over player positions and movement ([13]).”

Detecting *gamebots* has already been researched in MMORPG and it is not a primary concern in competitive genre. In the case of Extra-Sensory Perception (*ESP*) cheats, game companies have proposed solutions in various ways; from a policy-level solution that implements honeypots in a game-client ([47]), to a technical solution that limits a game server to send needless information that could be misused for providing additional information ([22]).

SYNOPTICON can be integrated with various cheating detection methods previously studied—such as statistics analysis, similarity analysis and machine learning—but it operates under the assumption that all users can evaluate each other mutually. For instance, vision-based methods, which detects cheating users by evaluating the information on the user’s screen, fall outside the scope, as they rely solely on data from a single user. Lastly, while SYNOPTICON can be applied across all competitive genres, for simplicity and consistency in terminology, the explanation in Section 3 will focus on its application to *aimbots*, a common type of scripting in FPS genre.

3 Synopticon: consensus-based cheating detection system for competitive games

3.1 Overview

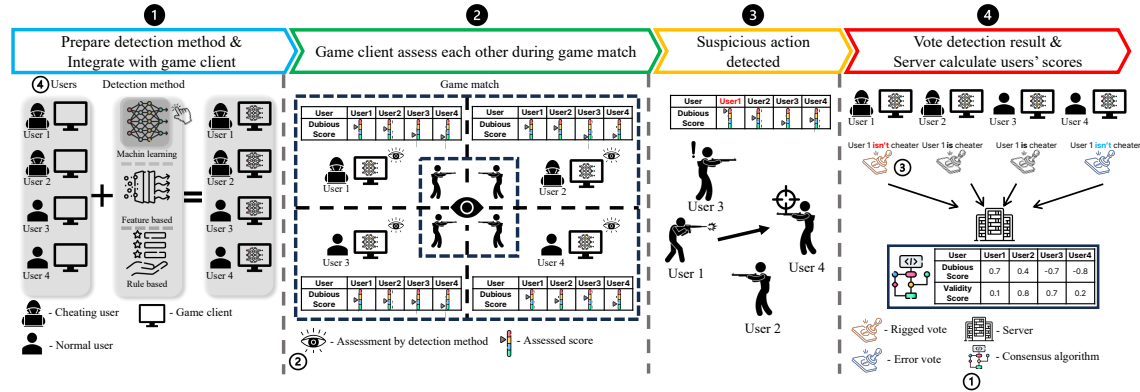


Fig. 1. Overall design of SYNOPTICON: the design and concept is described in Section 3.1—①,②,③,④, and the basic requirements is discussed in Section 3.4—①,②,③,④

The SYNOPTICON assumes a general structure of online competitive games, that comprises users who play a game and a game server that supervises a game. To play a match, users request match-making from the server, and the server sets a match which is generally made up of competitions between two groups of users (e.g., N vs N). During the match, the server receives data from client-side (e.g., keyboard-inputs for movement and mouse-inputs for gunfire), and the server continuously calculates and updates the state of the game. Then, the server broadcasts the updated state to users, and the state is rendered by a game-client at client-side (e.g., movement of characters and graphical effects of gunfire). During this process, SYNOPTICON works as shown in Figure 1.

① Detection method is a method for detecting cheating users. For instance, one of many client-side methods suggested in previous studies can be used as a detection method, regardless of its approach—such as machine learning, statistics

analysis and rule-based detection—as long as it can evaluate user’s behavior by using data received from the server. The detection method is implemented in a game-client and monitors users’ behavior during a match using the received data.

② Information from the server contains not only one player’s data but also other players’ data nearby, so the game-client can render the states of a game. For example, the game-client receives the states—such as character’s movement by keyboard-inputs and gunfire by mouse-inputs—from the server and renders all characters’ movements nearby, and graphical effects of flames and explosion of the gunfire in battlefield. By using these sets of data, the detection method in each user’s game-client can evaluate each other’s behavior during a match.

③ If any detection method in each user detects an abnormal (when a suspicious score exceeds a threshold), a *vote* begins. Each user casts their evaluation results as a ballot to the server. One ballot contains information about whether one user is a cheating user or a normal user.

④ Based on the users’ ballots, the server uses a consensus algorithm to calculate each user’s score and differentiates between cheating users and normal users after a match.

To sum up, SYNOPTICON consists of two main components: the detection method on client-side and the consensus algorithm on server-side. The details of each will be described in the following sub-sections.

3.2 (Client-side) Voting initiator with appropriate detection methods

SYNOPTICON assumes two types of users, *cheating* users and *normal* users. All users play a game using a game-client that contains appropriate detection methods. Therefore, *vote* can occur when the detection method identifies abnormal actions of a cheating user. If the method successfully distinguishes between cheating and normal users, each user will cast *True* for a cheating user and *False* for a normal user to the server.

Besides accurate evaluations, there may be two additional scenarios of mis-estimation. First, users might cast an *error vote* when the evaluation of the method is wrong. Secondly, cheating users can manipulate their ballot and send a *rigged vote* to the server, as the client-side methods are vulnerable to tampering by cheating users. However, the *rigged vote* by normal users is not considered.

3.3 (Server-side) Cheater discriminator with a proposed consensus algorithm

The server evaluates users’ scores using a consensus algorithm for each *vote* that occurs during a match. In our design, we introduced two scores, **Validity** and **Dubious**, for the evaluation.

At the beginning of each match, both scores are set to initial state for all users. To clarify, since no information about users is given to the server, all users have the maximum **Validity** score of 1 and the neutral **Dubious** score of 0. The server updates both scores for each *vote* based on the **Ground truth** decided by users’ ballots for each *vote*. The server sets the **Ground truth** according to the majority of ballots, since the server doesn’t know who is a cheating user. The *vote* assumes a secret vote, in which one’s poll is concealed from others. This assumption is reasonable; because, matches between users are randomly set, and it is unlikely that they know each other and cooperate to manipulate the *vote*.

The two scores are made by a proposed consensus algorithm on server-side and its details are on [Algorithm 1](#). Two factors were considered for the design of the algorithm. First, a trustworthy user affects more on the *vote*. Depending on the **Validity** score, users with higher scores influence more on the evaluation of the **Dubious** score, while those with lower scores have less impact. Second, we added a voting history for the evaluation of **Validity**. It is to take the *error vote* occurred by wrong evaluation into consideration. Users, who cast the vote consistent with the **Ground truth**

Algorithm 1: Validity, Dubious score evaluation

GT: Ground truth
Users: List of users
 W_v : Weight for Validity
 W_h : Weight for History
 Q_N : Queue for voting record of userN
 D_N : Dubious score of userN
 V_N : Validity score of userN ($0 \leq V_N \leq 1$)
 $Vote_{target}(N)$: A vote of userN for target
Function score_evaluation($target_user, GT$):

```

    foreach user in the Users do
      t_cnt ←  $Q_{user}.count(True)$ ;
      f_cnt ←  $Q_{user}.count(False)$ ;
      hist_record ← t_cnt - f_cnt;
      if  $Q_{user}.length() > 10$  then  $Q_{user}.deque()$ ;
      if  $Vote_{target\_user}(user) \neq GT$  then
        user_lied ← True;
      else
        user_lied ← False;
      if user_lied is True then
         $V_{user} \leftarrow V_{user} - W_v + hist\_record * W_h$ ;
         $Q_{user}.enqueue(False)$ ;
      else
         $V_{user} \leftarrow V_{user} + W_v + hist\_record * W_h$ ;
         $Q_{user}.enqueue(True)$ ;
       $D_{target\_user} \leftarrow D_{target\_user} + V_{user}$ ;
  
```

Function do_vote($Users, GT$):

```

    foreach target_user in the Users do
      score_evaluation(target_user, GT)
  
```

frequently, get less penalty on their **Validity**, even if they vote incorrectly. In contrast, more penalties will be imposed on users who lie often, and, as a result, they will lose their influence on the *vote* rapidly, with their decreasing **Validity**.

In the end, SYNOPTICON considers users with high **Dubious** score as cheating users. This design allows SYNOPTICON to detect cheating users, while effectively keeping the overhead on server-side low. At the same time, it successfully prevent the influence of **rigged vote** on the *vote* by cheating users who are potential adversaries of client-side methods and are able to tamper, repudiate, and manipulate the results. For example, if cheating users do vote honestly, they have to admit that they are cheaters. If they do **rigged vote** and say lie against the **Ground truth** to repudiate charges on them, their **Validity** would decrease and lose their influence on the *vote*. Ultimately, cheating users must either admit to cheating or lie against the **Ground truth**, which increase their **Dubious** scores or decrease their **Validity** scores.

However, as mentioned above, we assumed that an appropriate existing detection method is available, so if the detection method cannot distinguish between cheaters and normal users well enough, SYNOPTICON will not work properly. Therefore, the following subsection addresses some additional research questions to determine the detailed operational parameters for the proposed system to work in practice.

3.4 Operational parameter testing with simulation

To assess the effectiveness of SYNOPTICON as a cheating detection system and to determine the minimum requirements for its successful operation, we conducted a series of experiments with simulation. These experiments were designed to address the following four questions and associated critical factors:

Q1-1 Will the consensus algorithm work?

- ① Final dubious score of users after each match

Q1-2 How much of *model acc* is needed for accurate detection?

- ② Least accuracy of detection method for good-enough performance

Q2-1 How powerfully can cheating users influence the *vote*?

- ③ Influence of **rigged vote** by cheating users

Q2-2 How many cheating users can be endured?

- ④ Maximum ratio of cheating users that can be handled

The simulation comprises multiple users and a server, and there are four controllable variables as follows: *vote count*, the number of votes occurred during one match. *benign num*, the number of normal users in a match. *cheater num*, the number of cheating users in a match. *model acc*, the accuracy of the detection method. We performed two experiments by changing the variables to answers **Q1-1** and **Q1-2** respectively. In addition, for **Q2-1** and **Q2-2**, we reflected the **rigged vote** to the simulation. For that, we supposed that cheating users can behave tactically. Hence, we carried out each experiment on three different conditions: *Without liar* that no users poll the **rigged vote**. *Random liar* that cheating users can manipulate the ballot and do **rigged vote** for one-half of their ballot—for left one-half, however, they simply poll honestly. *Tactical liar* that a cheating user always votes on not cheater (*False*) for oneself. Every match in the simulation is independent mutually; in other words, all users' scores are reset to the initial state for every match.

3.4.1 Answer to Q1-1 and Q2-1. In the first simulation, we verified that our consensus algorithm actually works by recording the final **Dubious** score of users after each match. The simulation supposed two normal users and one cheating user as players, and only one *vote count* in a match. The *model acc* was set to 80%. The reason for the small *vote count* and small number of users is to reflect the influence of the *model acc* and to reduce the compensations from the voting history record in our consensus algorithm. Because, as more voters participate in a vote and as more ballots are cast, the results will be stabilized, yet we want harsher condition for the simulation.

After several matches, we analyzed the median value of the final **Dubious** scores after matches by using the box plots. In addition, to examine the correlation between *model acc* and the final **Dubious** score, we observed shifts in the median scores of each two groups by changing the *model acc*. The result on [Figure 2-\(a\)](#) proves that scores between two groups are distinguishable, confirming that our consensus algorithm functioned as intended. [Figure 2-\(b\)](#) demonstrates that the tactics of cheating users had an effect. Especially on [-\(b\)-\(2\)](#) and [-\(b\)-\(3\)](#), a gap between two groups are more narrow compared to [-\(b\)-\(1\)](#). Moreover, the results underscores the importance of *model acc*, as the gap narrowed sharply in accordance with the decreasing of *model acc*.

3.4.2 Answer to Q1-2 and Q2-2. In the second simulation, we observed how much of the *model acc* is required to achieve the high accuracy of classification. In order to answer the question, we evaluated the *Accuracy* of the classification by considering users with **Dubious** scores below zero as normal users and those above zero as cheating users. In this simulation environment, we increased the total number of users to 100, in order to adjust the ratio between cheating and normal users.

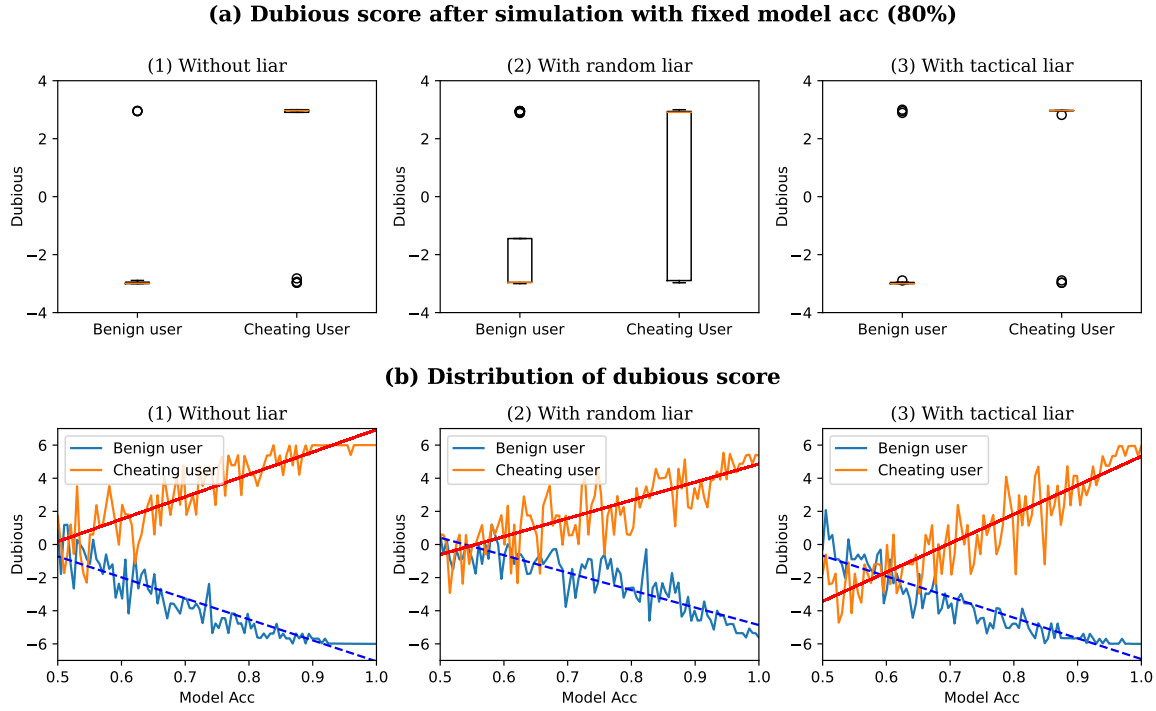


Fig. 2. Results of the first simulation. The proposed consensus algorithm was examined. **(a)** demonstrates the **Dubious** scores of normal users and cheating users at fixed 80% of *model acc*. **(b)** demonstrates the scores of cheating and normal users at different *model acc* from 50% to 100%

Simulation results on Figure 3-(a) indicate that an approximate 80% of the *model acc* is required to ensure the roughly 100% of *Accuracy*, when the ratio of cheating users fixed on 33%. Furthermore, we also examined how the *Accuracy* showed on Figure 3-(a) changes depending on the ratio of cheating users, from 0% to 50%. The results on Figure 3-(b) illustrates that if *model acc* satisfies more than 80%, SYNOPTICON can assure nearly 100% of *Accuracy* under the condition of roughly 40% of cheating users. On the other hand, it is hard to imagine the case that cheating users account for more than 40% among all users. For example, in the case of (5 vs 5) competitive games like ‘Counter-Strike: Global Offensive’, even if only 7% of the total users are using cheats, there is approximately a 50% chance that there will be at least one cheating user in the match ([28]), proving that such high rate is unrealistic. If it exists, otherwise, it is unlikely to consider that such games are properly managed.

In addition, results on Figure 3-(a)-(3) and Figure 3-(b)-(3) suggest that the tactics of liar worked less effectively than in the first simulation, and it shows that increased number of voters and ballots effects the stability of results. However, the tactic of Random liar still worked, and the results on Figure 3-(b)-(2) indicates that SYNOPTICON becomes ineffective when 50% of users are cheating users; because, in such a situation, the **Ground truth** can be manipulated by cheating users.

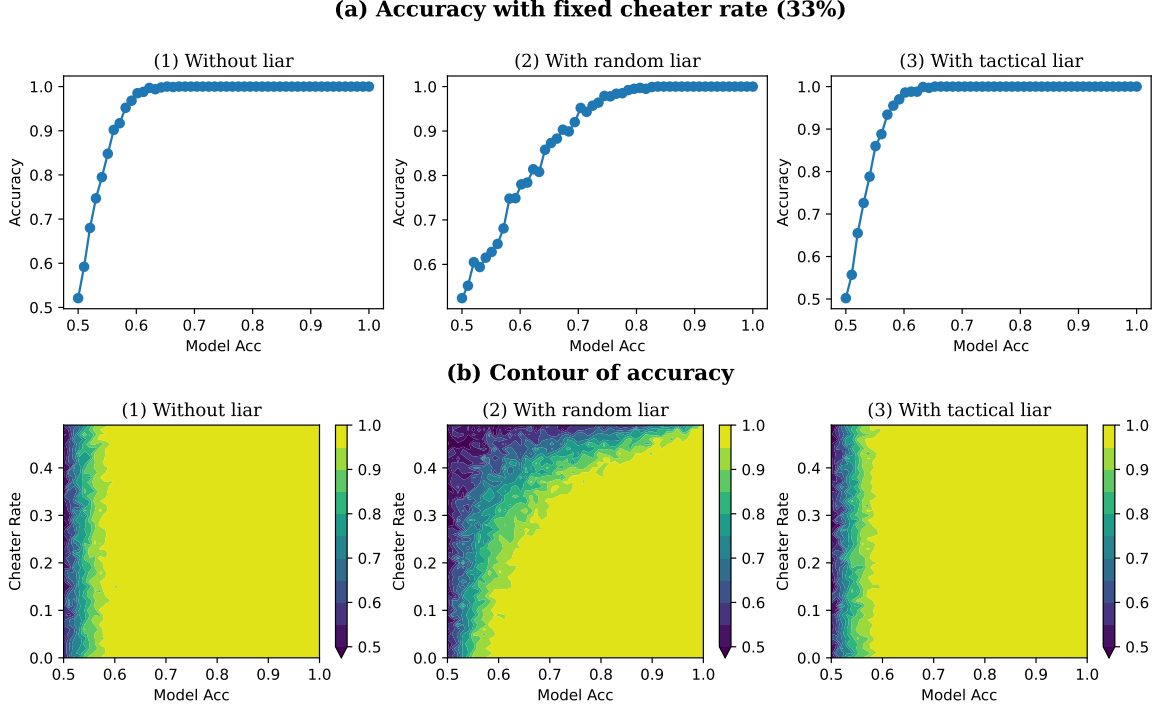


Fig. 3. Results of the second simulation. The minimum *model acc* was examined. **(a)** demonstrates the needed accuracy at a fixed cheater rate of 33%. **(b)** demonstrates the changing accuracy at different cheater rates from 0% to 50%

4 Evaluation

In this section, we apply SYNOPTICON to a public dataset of an online game named ‘Counter-Strike: Global Offensive’ for the purpose of evaluation.

4.1 Dataset description

The dataset was released by [7]. The research focused on client-side detection method for *aimbots*. They employed a variety of users, from professional players to average players, and recorded each player’s game-playing logs at client-side. The players played for several matches, and, before each match, a subset of players was asked to use *aimbots*. Since their method was applied on client-side, the log contains the data received from a server. Specifically, the data contains timestamp, player’s current position (x, y, z), player’s aiming angle (ax, ay, az) and in-game event (fire, hit, dead). As a detection method, they employed a Recurrent Neural Network (RNN), one of the machine learning techniques, to differentiate cheating users. Because of supports from *aimbots*, cheating user’s actions are different from those of normal user, and the method can detect cheating users by detecting such differences. For example, cheating user’s aiming angle may change so rapidly that even professional players can’t imitate it.

For evaluation, we preprocessed the dataset and customized our consensus algorithm considering the characteristics of the FPS genre. According to the authors of the dataset ([7]), their detection method reached more than 90% of accuracy, which meets our minimum requirement of more than 80% *model acc* for almost 100% of *Accuracy*.

4.2 Preprocessing of dataset

First, the overall timeline of a match has been reconstructed by integrating every user's log and has been divided into small timelines. The smaller timelines have been divided based on a battle between users; because, in most cases, *aimbots* is used for a battle. The slide window of the battle has been sliced considering the timestamps when fire events were recorded. After constructing battle timelines of a match, a group of users, who engaged in a battle, has been extracted for each battle.

Second, for each battle, the detection method evaluated logs of each participant in a battle, which includes the information of other users received from a server. After each battle, the ballots (evaluation results) of each user are organized, and the server evaluates them and calculates the two scores for each user based on the consensus algorithm.

4.3 Customization of consensus algorithm

We customized the consensus algorithm considering the characteristics of FPS.

Real-time battle Since the battles occur sporadically, only a subset of users participates in a battle, not all users. Therefore, when *vote* is initiated, voters are limited to users who participated in a battle.

Constant changing of users During a battle, a number of users participating in a battle is constantly changing, due to death in combat or simply leaving a battlefield. In other words, a number of voters changes continually. However, users are not able to evaluate the user who killed early and vice versa. Thus, we limited voting rights to the users who participated in a battle long enough, so the method in each user could evaluate others. In addition, we also changed *target_user*, who is being evaluated by others, from all users to users who are detected by the detection method.

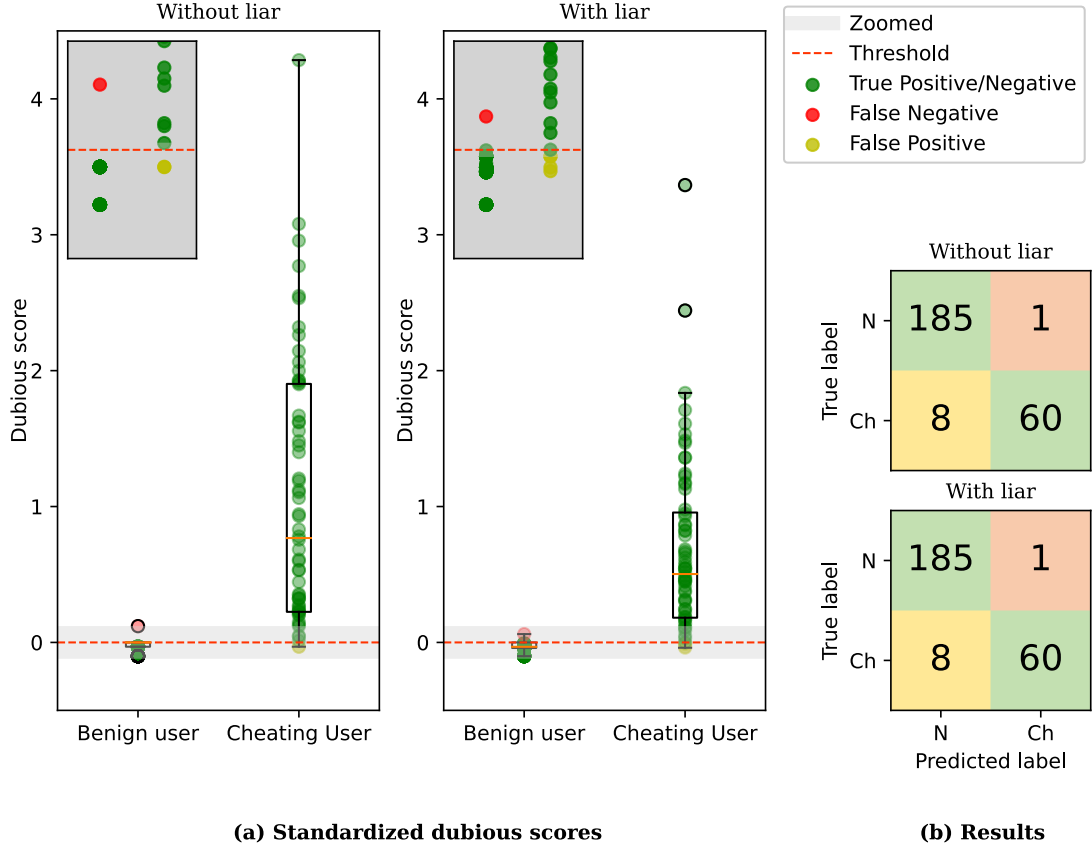
Self voting Limiting and changing voters could have a negative impact for some cases. One example is when one normal user combats with one opposing cheating user (1 vs 1). If the ratio between cheating and normal users is set to fifty-fifty, the **Ground truth**, decided by the majority, could be manipulated by cheating users. Considering this, a vote for oneself was excluded. In addition, to prevent the intentional lie of cheating users targeting normal users, the *vote* only occurs when more than two users participated and **Dubious** score was updated when the majority of users agreed that the *target_user* is a cheater.

A customized algorithm for the FPS genre can be found on [Algorithm 2](#) in [Appendix A](#).

4.4 Experimental design

In previous research, 28 matches have been played, and logs of each player during a match have been recorded in the dataset ([7]). The RNN model had been trained and tested on randomly divided logs of matches. We followed the process of previous research and split the logs based on match-basis (Note that it was named Game-based Split in the previous study). More specifically, this study performed k -fold cross-validation, which uses $k - 1$ sets of data for the training of a model and 1 set of data for the test of the trained model. In previous research, the k was set to 7, since the divided sets could have the identical number of matches.

For each validation, we checked the final **Dubious** scores of users and determined a threshold that maximizes the *Accuracy*, since game companies more worry about false alarms, such as **FN** and **FP**. (When a normal user is mistaken for a cheating user, and when a cheating user is mistaken for a normal user, respectively). Regarding the tactics of the liar, we adopted the *Random liar*, since it has proved to be a harsher condition for SYNOPTICON as shown on [Figure 3](#).



N: normal user Ch: cheating user

Fig. 4. Results of experiment. (a) demonstrates the standardized final **Dubious** scores of all validations and box plots of the final scores. To fix a threshold at 0 across all validations, for each validation, a threshold was subtracted from final scores. (b) is the confusion matrix of the results.

4.5 Results

Figure 4-(a) shows standardized **Dubious** scores of all users for all validations, and Figure 4-(b) demonstrates the overall experimental outcomes. We standardized the results of each validation by fixing a threshold at zero across all validations. Specifically, for each validation, we subtracted the determined threshold that maximizes the *Accuracy* from the final **Dubious** scores to fix the threshold at zero across all validations.

Figure 4-(a) demonstrates that the tactics of liar worked to some extent, since the range of **Dubious** scores has been shortened. Still, as shown in Figure 4-(b), SYNOPTICON effectively differentiates normal users from cheating users, regardless of the presence of liar. However, the results appear to show some false alarms. To identify the underlying causes, we conducted in-depth analysis of the false cases and found that the primary cause of false alarms was a low *vote count*. The *vote count* for each false case is on Table 3. The table shows that the *vote* rarely occurred and fewer than one *vote* was performed during a match, whereas an average *vote count* for cheating users was five. Considering the

Table 3. Total number of votes for each match for which false alarms occurred

Result	Users		Votes	
	Total user	Cheater	Target user	Vote count
FN	10	4, 6	9	1
FP	8	3, 4	3	0
FP	8	2, 5	5	1
FP	8	1, 4	1	0
			4	0
FP	10	1, 9	9	0
FP	10	0, 8	0	0
FP	10	1, 5	5	0
FP	10	1, 7	1	0

results of the simulation on Figure 2 and Figure 3 in Section 3, limited number of *vote* and voters can negatively impact the stability of results.

On the other hand, the reasons for the small *vote count* were various. In most **FP** cases, because of an early death and leaving battlefield, cheating users participated in a battle too shortly, so other users can't evaluate them properly. Furthermore, limited number of large-scale battle, in which many users participated, contributed to the low *vote count* and wrong evaluation. On the other hand, in **FN** cases, the primary issue was the accuracy of the detection method.

5 Application and discussion

Even though SYNOPTICON has been demonstrated its usability through simulation and its effectiveness with a real game dataset, the experiments mentioned above are limited to a game-play and also performed on a match-basis. Moving forward, for **Ch4**, we need to address the challenge of collecting reliable datasets from reliable sources. In addition, despite its usability, SYNOPTICON should solve the "50% problem" of which the ratio of cheating users is more than 50%, highlighted by the results on Figure 3. We discuss these issues in this section. We carried out an additional simulation with our virtual game simulator based on a real-like environment. We implemented some operational policies and observed how we could distinguish cheating users from normal users to obtain reliable data. Finally, this section shows how well-developed policies can provide solutions to the seemingly unsolvable 50% problem.

5.1 Application on a real-like environment

We simulated two types of competitive games of N vs N and 1 vs 1 . The first game represents typical competitive games and their settings, such as MOBA. For example, many of MOBA games follow the 5 vs 5 setting including the 'League of Legends' of Riot Games and 'Dota2' of Valve. The second game is for other genres, such as Action and RTS. For example, many action games follow the 1 vs 1 setting, such as 'Tekken' of Bandai Namco and 'Street Fighter' of CAPCOM. It is worth noting that the settings of the second game type could give hints about how to overcome the inherent limitation of SYNOPTICON on which 50% of users are cheating users; because, in 1 vs 1 games only one **rigged vote** by one cheating user can hold a majority and makes it hard to set the **Ground truth**.

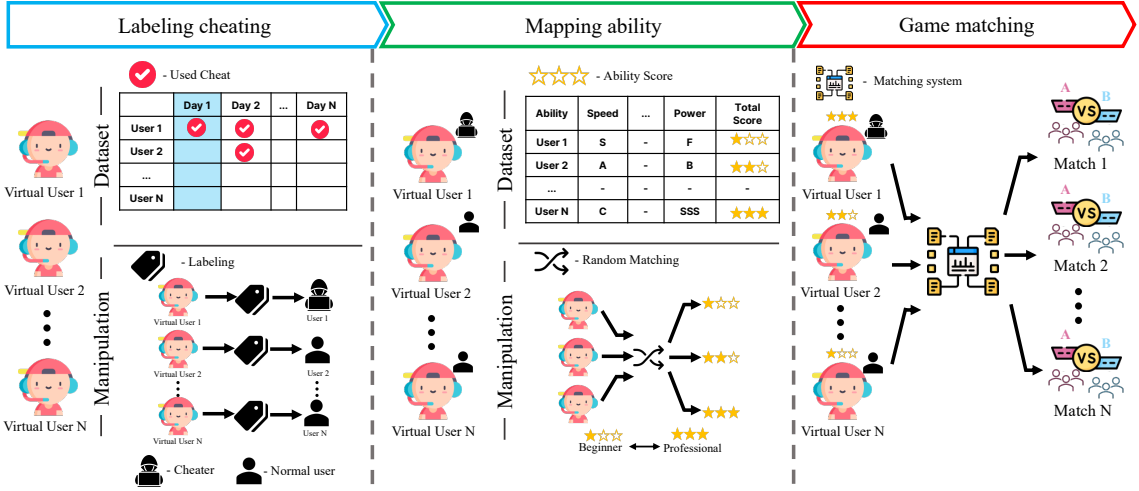


Fig. 5. Simulation setting for a real-like online game. A detailed explanation and the dataset used for each step can be found in Section 5.1.1

5.1.1 Simulation description. In order to make our virtual game simulator as realistic as possible, which reflects real-world conditions, it was designed considering two factors: changing number of cheating users and the characteristics of a realistic online competitive game service. For the first factor, it was necessary to consider users' cheating usage patterns; because, several external elements in the real-world influence the users' cheating usage patterns. For example, one day, a normal user could start to use cheating, by watching one of their friends is using cheating or by yielding to the temptation of using cheating. Second factor is to reflect the characteristics of competitive games in the real-world. Competitive games are played on match-basis, and each match is a competition between two groups of users (or two users) with similar game-play abilities. To broaden our discussion from individual matches to overall game service and policy establishment, a realistic match-making system imitating a real-world game is essential.

To satisfy requirements, we integrated two datasets from studies by [9] and [48] into our simulation. Each steps for the integration is illustrated on Figure 5. In the first step, we created virtual users and each of them is labeled with the cheating usage pattern of the real-world users. The dataset was initially published by [48]. Their study examined the social contagion of cheating behavior found in social networks of online games by using bot detection logs of the real-world game named 'Aion' developed by NCSoft. Because this dataset reflects real-world factors influencing users' cheating usage pattern, our simulation can help policymakers develop operational policies that can be applied in real-world scenarios. Second, to accurately simulate a real-world game environment, we incorporated a match-making system and an ability dataset of real-world users from the study by [9]. The virtual users were randomly assigned ability scores, and matches were organized using the match-making system. Their study proposed a match-making system with efficiency and high performance. Most importantly, their method had been applied to the real-world games named 'Fever Basketball' and 'Justice Online' by Netease.

Our system simulates matches on a daily basis with ten thousand of virtual players participating in a certain number of matches per day, from one to nine. For the simulation of the first game with N vs N setting, the number of N was set to 3 and each user's **Validity** and **Dubious** scores were randomly matched with one of the scores in Section 4.5 after each match, according to the label attached at the first step, whether a user is a cheater or not. Since the user ability

dataset only contains one thousand users' data, we employed over-sampling by randomly selecting and averaging the abilities of two users. The second game, simulating *1 vs 1* games, follows the same setup as the first game, except that the simulator used in Section 3 is introduced to access users' two scores after each match. In a similar flow of Section 3, we set the variables: *benign num*(1), *cheater num*(1), *model acc*(80%), and *vote count* has been set randomly between 0 to 5. For the same reason in previous section, we adopted the *Random liar* as a tactic.

5.1.2 Policy settings and implications. For both games, we applied a policy for user evaluation and studied the results. Afterwards, we then explored potential policies that could be implemented to detect cheating users.

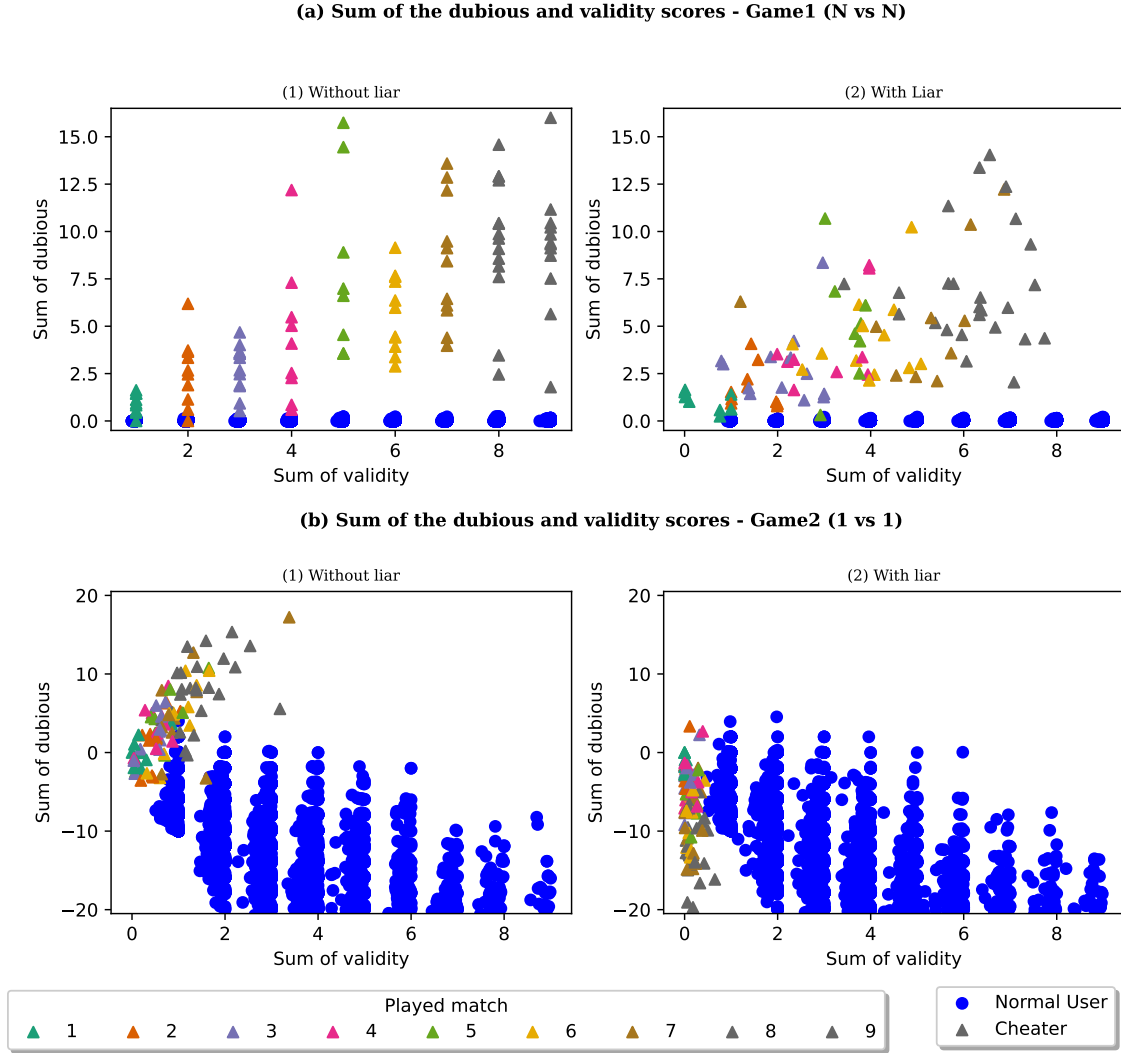


Fig. 6. Result of the first policy. Users' **Dubious** and **Validity** scores have been added. (a) demonstrates the results of Game1 (*N vs N*). (b) demonstrates the results of Game2 (*1 vs 1*)

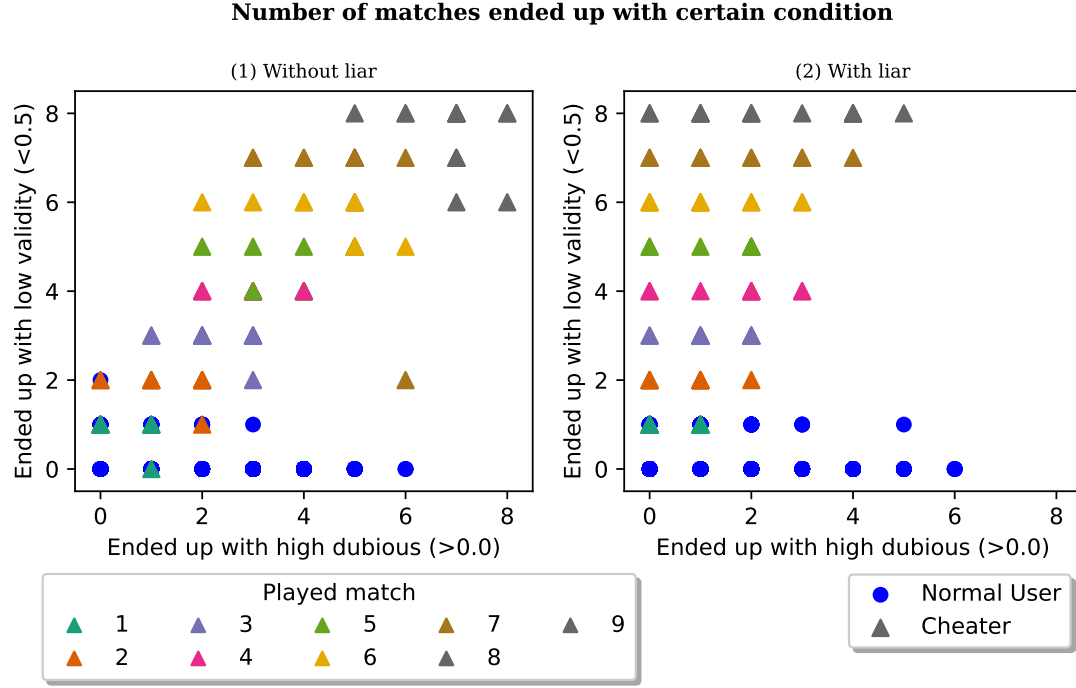


Fig. 7. Result of the second policy implemented on Game2 (1 vs 1). The number of matches has been counted when a user satisfied specific conditions. The conditions are low **Validity** score (lower than 0.5) and high **Dubious** score (higher than 0.0)

Policy of adding. For first policy, the evaluation proceeds by calculating the sum of the two scores each, after matches. Figure 6 presents the overall scores after a one-day simulation. Figure 6-(a) demonstrates that even the simple policy of adding scores can effectively differentiate cheating and normal users based on the sum of the **Dubious** score. However, Figure 6-(b) indicates that simply adding scores does not work efficiently for the second game.

Policy of counting. We addressed the problem identified on Figure 6-(b) by leveraging the power of the majority. That is to say, if our consensus algorithm functions correctly, two normal users playing a match will both have high **Validity** and low **Dubious** scores. In contrast, when a normal user faces a cheater, two scenarios may occur: If the cheating user lies, both users will have low **Validity** and high **Dubious** scores. If the cheating user is honest, the cheating user will have high **Validity** and high **Dubious** scores, while a normal user will have high **Validity** and low **Dubious** scores.

Based on this understanding, two players will have low **Validity** scores, if someone lies, and cheating users will have high **Dubious** scores if they don't lie. Therefore, we counted the number of matches when users satisfied certain conditions: when it has high **Dubious** scores and low **Validity** scores each. As a result, cheating users will be counted more frequently, and counted number of normal users will be distributed until the normal users take a majority in all users. The result on Figure 7 demonstrates that cheating users can be filtered much clearly even in the 1 vs 1 games. However, this strategy takes effect as far as normal users secure a majority in overall user-pool.

5.2 Discussion on applicability

As previous sections present, implementing effective strategies can provide solutions for sustainable operation (**Ch4**) and help policymakers ensure the effective use of SYNOPTICON (**Ch3**)—across various scenarios, such as *1 vs 1*. The policymakers can collect reliable data from users to re-adjust detection methods and quickly ban cheating users. However, the two achievements require rapid detection speed (**Ch1**). Let us say, if it takes one month, for example, policymakers may struggle to obtain reliable data from users; because, the data could be adulterated if one normal user starts to use cheating during the collecting. Consequently, a delayed ban may frustrate not only the policymakers but also users.

Moreover, more sophisticated policies may be needed. For instance, different detection methods may have to be applied to different user-pools depending on average abilities of users in each user-pool. If the gap of the abilities between professional and average players is huge, a detection method optimized for average users might mistakenly flag professional players as cheaters. In conclusion, despite the progress made in this section, we acknowledge that our approaches may be insufficient for some games. Therefore, in addition to SYNOPTICON, we open our datasets and code, so others can test their advanced detection method and policies on our simulator. The sources are [available here](#).

6 Related work

Server-side *aimbots* detection based on user behavior. Previous studies focusing on *aimbots* adopted various methods for cheating detection. [26] and [3] differentiate cheating users with Support Vector Machine (SVM) classifiers. [15] compared five machine learning methods for detection—including Decision trees, Naive Bayes, Random forests, Neural networks and SVM—and compared the results of detection. They evaluated the playing-log stacked on server-side received from client-side with statistics analysis and machine learning methods, yet server-side approaches are suffered from high overhead.

Client-side *aimbots* detection based on user behavior. [42] conducted case studies of 9 experiments by comparing the results applied 3 detection methods (Decision Tree, SVM, Naive Bayes) to 3 datasets from different sources (Mouse input, Keyboard input, Game events), and provide valuable insights for choosing of detection method. In addition, [36] used a machine learning method and evaluated keyboard-, mouse- inputs. Likewise, [7] applied their RNN model for detection, and they used the character’s coordinate and aiming angle as input data. Even though their client-side approaches achieved impressive results, the problem of low applicability is to be solved. However, SYNOPTICON can be integrated with their prestigious research and will solve the problem.

Other approaches for cheating detection. As different approaches, [18] introduced statistics analysis. They use various features for detection, such as winning rates and play-time. [51, 52] assessed user input, such as cursor movement and cursor acceleration. The statistical approach may not perfectly fit with SYNOPTICON, as it takes a relatively long time and is proven less accurate. Still, in the operational aspect, it would give meaningful insights for policymakers. To achieve this, our real-like simulation can be applied. [10] and [19] adopted network-based approach. They analyzed encrypted network traffic by using the machine learning method. Even though their approach may differ from approaches based on user behavior, network-based approaches can also be applied to SYNOPTICON as well; because, our basic assumption is based on using data received from a server.

7 Conclusion and Perspective

Cheating detection in competitive games got less attention from the academy and industry as well. Even for its increasing size in the gaming market, it was less attractive for researchers, as it requires sophisticated detecting methods and datasets. Existing studies, also, suffered from several challenges including low applicability and high overhead. In this work, we addressed those challenges that have been faced in previous studies. At first, we discussed that applying a detecting method on server-side could incur a considerable cost of overhead, yet if we apply the method on client-side, there is also a chance of tampering by clients, which is unrealistic to apply. Anti-cheat solutions, which is commonly used in the industry, also have problems in terms of user privacy. Next, we introduced SYNOPTICON. SYNOPTICON assumes that the detection method was applied to all users, so server-side can keep overhead low. Tampering by users is protected by normal users' consensus. Lastly, SYNOPTICON does not require personal information of users, as it only uses data received from server-side. In discussion, we practiced that SYNOPTICON can be applied to real-world games and showed that it successfully worked as a cheating detection system.

Future research on cheating detection should focus on more various competitive genres, such as MOBA and Action, as cheating detection methods regarding these are not addressed yet, to the best of our knowledge. Furthermore, while our experiments proved the effectiveness of policy setting, studies on cheating detection methods of competitive games are required to offer more insight into operational aspects of their method. Especially on competitive games, discussions about operation and managing are insufficient.

References

- [1] Activision Blizzard. 2024. RICOCHET Anti-Cheat™: Call of Duty's Anti-Cheat Initiative. <https://support.activision.com/articles/ricochet-overview> visited on 2024-8-14.
- [2] Rohan Aggarwal. 2023. Bypassing Anti-Cheats and Hacking Competitive Games. <https://www.blackhat.com/asia-23/briefings/schedule/#bypassing-anti-cheats--hacking-competitive-games-30572>
- [3] Hashem Alayed, Fotos Frangoudes, and Clifford Neuman. 2013. Behavioral-based cheating detection in online first person shooters using machine learning techniques. In *2013 IEEE conference on computational intelligence in games (CIG)*. IEEE, IEEE, 1–8. <https://doi.org/10.1109/CIG.2013.6633617>
- [4] Amazon Web Services, Inc. 2024. Epic Games on AWS. https://aws.amazon.com/solutions/case-studies/innovators/epic-games/?nc1=h_ls visited on 2024-8-14.
- [5] BBC. 2019. The cheat hackers ruining' gaming for others. *BBC* (2019). <https://www.bbc.com/news/av/technology-49740155> visited on 2024-8-14.
- [6] Jeremy Bentham. 1791. *Panopticon or the inspection house*. Vol. 2.
- [7] Minyeop Choi, Gihyuk Ko, and Sang Kil Cha. 2023. BotScreen: Trust Everybody, but Cut the Aimbots Yourself. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 481–498. <https://www.usenix.org/conference/usenixsecurity23/presentation/choi>
- [8] Mia Consalvo. 2009. *Cheating: Gaining advantage in videogames*. mit press. <https://doi.org/10.7551/mitpress/1802.001.0001>
- [9] Qilin Deng, Hao Li, Kai Wang, Zhipeng Hu, Runze Wu, Linxia Gong, Jianrong Tao, Changjie Fan, and Peng Cui. 2021. Globally Optimized Matchmaking in Online Games. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (Virtual Event, Singapore) (KDD '21)*. Association for Computing Machinery, New York, NY, USA, 2753–2763. <https://doi.org/10.1145/3447548.3467074>
- [10] Bo Dong, Md Shihabul Islam, Swarup Chandra, Latifur Khan, and Bhavani Thuraisingham. 2018. GCI: A transfer learning approach for detecting cheats of computer game. In *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, IEEE, 1188–1197. <https://doi.org/10.1109/BigData.2018.8621959>
- [11] Christoph Dörner and Lukas Daniel Klausner. 2024. If It Looks Like a Rootkit and Deceives Like a Rootkit: A Critical Examination of Kernel-Level Anti-Cheat Systems. In *Proceedings of the 19th International Conference on Availability, Reliability and Security (Vienna, Austria) (ARES '24)*. Association for Computing Machinery, New York, NY, USA, Article 62, 11 pages. <https://doi.org/10.1145/3664476.3670433>
- [12] Electronic Arts Inc. 2022. A Deep Dive on EA anticheat for PC. <https://www.ea.com/en-au/security/news/eaac-deep-dive> visited on 2024-8-14.
- [13] Epic Games. 2024. Anti-Cheat Interfaces. Available from Developer Resources of Epic Games. <https://dev.epicgames.com/docs/game-services/anti-cheat/anti-cheat-interfaces> visited on 2024-8-14.
- [14] Epic Games. 2024. How do I install Easy Anti-Cheat on my computer to play Fortnite? https://www.epicgames.com/help/en-US/c-Category_Fortnite/c-Fortnite_TechnicalSupport/how-do-i-install-easy-anti-cheat-on-my-computer-to-play-fortnite-a000086154 visited on 2024-8-14.
- [15] Luca Galli, Daniele Loiacono, Luigi Cardamone, and Pier Luca Lanzi. 2011. A cheating detection framework for unreal tournament iii: A machine learning approach. In *2011 IEEE Conference on Computational Intelligence and Games (CIG'11)*. IEEE, IEEE, 266–272. <https://doi.org/10.1109/CIG.2011.6088888>

- 11.6032016
- [16] Nicilas Guigo and Joel St John. 2014. Next level cheating and leveling up mitigations. <https://www.blackhat.com/docs/asia-15/materials/asia-15-StJohn-Next-Level-Cheating-And-Leveling-Up-Mitigations.pdf>
 - [17] Mee Lan Han, Byung Il Kwak, and Huy Kang Kim. 2022. Cheating and detection method in massively multiplayer online role-playing game: systematic literature review. *IEEE Access* 10 (2022), 49050–49063. <https://doi.org/10.1109/ACCESS.2022.3172110>
 - [18] Mee Lan Han, Jung Kyu Park, and Huy Kang Kim. 2015. Online game bot detection in fps game. In *Proceedings of the 18th Asia Pacific Symposium on Intelligent and Evolutionary Systems-Volume 2*. Springer, Springer International Publishing, Cham, 479–491. https://doi.org/10.1007/978-3-319-13356-0_38
 - [19] Md Shihabul Islam, Bo Dong, Swarup Chandra, Latifur Khan, and Bhavani Thuraisingham. 2022. GCI: A GPU-Based Transfer Learning Approach for Detecting Cheats of Computer Game. *IEEE Transactions on Dependable and Secure Computing* 19, 2 (2022), 804–816. <https://doi.org/10.1109/TDSC.2020.3013817>
 - [20] Sanghoon Jeon and Huy Kang Kim. 2021. TZMon: Improving mobile game security with ARM trustzone. *Computers & Security* 109 (2021), 102391. <https://doi.org/10.1016/j.cose.2021.102391>
 - [21] Denis Kotkov, Gaurav Pandey, and Alexander Semenov. 2018. Gaming bot detection: A systematic literature review. In *Computational Data and Social Networks: 7th International Conference, CSoNet 2018, Shanghai, China, December 18–20, 2018, Proceedings 7*. Springer, Springer International Publishing, Cham, 247–258. https://doi.org/10.1007/978-3-030-04648-4_21
 - [22] Krafton Inc. 2024. Dev Letter: Anti-ESP - NEWS. <https://pubg.com/en/news/6957> visited on 2024-8-14.
 - [23] Evan Lahti. 2018. Valve has 1,700 CPUs working non-stop to bust CS:GO cheaters. *PC Gamer* (March 2018). <https://www.pcgamer.com/vacnet-csgo/> visited on 2024-8-14.
 - [24] Eunjo Lee, Jiyoung Woo, Hyoungshick Kim, Aziz Mohaisen, and Huy Kang Kim. 2016. You are a Game Bot!: Uncovering Game Bots in MMORPGs via Self-similarity in the Wild.. In *23rd Annual Network and Distributed System Security Symposium, NDSS 2016 (23rd Annual Network and Distributed System Security Symposium, NDSS 2016)*. The Internet Society, 1–15. <https://doi.org/10.14722/ndss.2016.23436> Publisher Copyright: © 2016 Internet Society.; 23rd Annual Network and Distributed System Security Symposium, NDSS 2016 ; Conference date: 21-02-2016 Through 24-02-2016.
 - [25] Sung Je Lee, Eui Jun Jeong, Dae Young Lee, and Gyoung Mo Kim. 2021. Why do some users become enticed to cheating in competitive online games? An empirical study of cheating focused on competitive motivation, self-esteem, and aggression. *Frontiers in psychology* 12 (2021), 768825. <https://doi.org/10.3389/fpsyg.2021.768825>
 - [26] Daiping Liu, Xing Gao, Mingwei Zhang, Haining Wang, and Angelos Stavrou. 2017. Detecting passive cheats in online games via performance-skillfulness inconsistency. In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, IEEE, 615–626. <https://doi.org/10.1109/DSN.2017.20>
 - [27] Anton Maario, Vinod Kumar Shukla, A Ambikapathy, and Purushottam Sharma. 2021. Redefining the risks of kernel-level anti-cheat in online gaming. In *2021 8th International Conference on Signal Processing and Integrated Networks (SPIN)*. IEEE, IEEE, 676–680. <https://doi.org/10.1109/SPIN52536.2021.9566108>
 - [28] John McDonald. 2022. Robocalypse Now: Using Deep Learning to Combat Cheating in Counter-Strike: Global Offensive. <https://www.gdcvault.com/play/1024994/Robocalypse-Now-Using-Deep-Learning> visited on 2024-8-14.
 - [29] MITRE. 2005. CVE-2005-0295. Available from MITRE, CVE-ID CVE-2005-0295.. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2005-0295> visited on 2024-8-14.
 - [30] MITRE. 2022. CVE-2020-36603. Available from MITRE, CVE-ID CVE-2020-36603.. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-36603> visited on 2024-8-14.
 - [31] MITRE. 2022. CVE-2020-36603. Available from MITRE, CVE-ID CVE-2022-27095.. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-27095> visited on 2024-8-14.
 - [32] Stefan Mitterhofer, Christopher Kruegel, Engin Kirda, and Christian Platzer. 2009. Server-side bot detection in massively multiplayer online games. *IEEE Security & Privacy* 7, 3 (2009), 29–36. <https://doi.org/10.1109/MSP.2009.78>
 - [33] Gene Park. 2020. The lessons Riot learned from its Valorant Beta. *Washington Post* (2020). <https://www.washingtonpost.com/video-games/2020/06/02/valorant-launch-beta-lessons/> visited on 2024-8-14.
 - [34] Seonghyun Park, Adil Ahmad, and Byoungyoung Lee. 2020. Blackmirror: Preventing wallhacks in 3d online fps games. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 987–1000. <https://doi.org/10.1145/3372297.3417890>
 - [35] Cale J Passmore, Mathew K Miller, Jun Liu, Cody J Phillips, and Regan L Mandryk. 2020. A cheating mood: The emotional and psychological benefits of cheating in single-player games. In *Proceedings of the Annual Symposium on Computer-Human Interaction in Play (Virtual Event, Canada) (CHI PLAY '20)*. Association for Computing Machinery, New York, NY, USA, 58–70. <https://doi.org/10.1145/3410404.3414252>
 - [36] José Pedro Pinto, André Pimenta, and Paulo Novais. 2021. Deep learning and multivariate time series for cheat detection in video games. *Machine Learning* 110, 11 (2021), 3037–3057. <https://doi.org/10.1007/s10994-021-06055-x>
 - [37] Riot Games. 2018. WILL TRIBUNAL RETURN? <https://nexus.leagueoflegends.com/en-us/2018/08/ask-riot-will-tribunal-return/> visited on 2024-8-14.
 - [38] Riot Games. 2024. <https://hackerone.com/riot/?type=team> visited on 2024-8-14.
 - [39] Riot Games. 2024. /DEV: VANGUARD X LOL. <https://www.leagueoflegends.com/en-au/news/dev/dev-vanguard-x-lol/> visited on 2024-8-14.

- [40] Riot Games. 2024. What is Vanguard? <https://support.valorant.riotgames.com/hc/en-us/articles/360046160933-What-is-Vanguard> visited on 2024-8-14.
- [41] Hitomi Kimura Ryan Soliven. 2022. Ransomware Actor Abuses Genshin Impact Anti-Cheat Driver to Kill Antivirus. https://www.trendmicro.com/en_us/research/22/h/ransomware-actor-abuses-genshin-impact-anti-cheat-driver-to-kill-antivirus.html visited on 2024-8-14.
- [42] Ruan Spijkerman and Elizabeth Marie Ehlers. 2020. Cheat detection in a multiplayer first-person shooter using artificial intelligence tools. In *Proceedings of the 2020 3rd International Conference on Computational Intelligence and Intelligent Systems*. Association for Computing Machinery, New York, NY, USA, 87–92. <https://doi.org/10.1145/3440840.3440857>
- [43] Jianrong Tao, Jiarong Xu, Linxia Gong, Yifu Li, Changjie Fan, and Zhou Zhao. 2018. Nguard: A game bot detection framework for netease mmorpgs. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. Association for Computing Machinery, New York, NY, USA, 811–820. <https://doi.org/10.1145/3219819.3219925>
- [44] Paul Tassi. 2015. “League of Legends” Experiments With Near-Instant Ban System For Toxic Chat. *Forbes* (May 2015). <https://www.forbes.com/sites/insertcoin/2015/05/25/league-of-legends-experiments-with-near-instant-ban-system-for-toxic-chat/> visited on 2024-8-14.
- [45] Valve Cooperation. 2024. Overwatch FAQ. <https://blog.counter-strike.net/index.php/overwatch/> visited on 2024-8-14.
- [46] Valve Cooperation. 2024. Valve Anti-Cheat (VAC) System. <https://help.steampowered.com/en/faqs/view/571A-97DA-70E9-FF74> visited on 2024-8-14.
- [47] Valve Corporation. 2023. Cheaters Will Never Be Welcome in Dota. <https://www.dota2.com/newsentry/3677788723152833273> visited on 2024-8-14.
- [48] Jiyoung Woo, Sung Wook Kang, Huy Kang Kim, and Juyong Park. 2018. Contagion of cheating behaviors in online social networks. *IEEE Access* 6 (2018), 29098–29108. <https://doi.org/10.1109/ACCESS.2018.2834220>
- [49] Jiyoung Woo and Huy Kang Kim. 2012. Survey and research direction on online game security. In *Proceedings of the Workshop at SIGGRAPH Asia*. Association for Computing Machinery, New York, NY, USA, 19–25. <https://doi.org/10.1145/2425296.2425300>
- [50] Jiarong Xu, Yifan Luo, Jianrong Tao, Changjie Fan, Zhou Zhao, and Jiangang Lu. 2020. Nguard+ an attention-based game bot detection framework via player behavior sequences. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 14, 6 (2020), 1–24. <https://doi.org/10.1145/3399711>
- [51] Su-Yang Yu, Nils Hammerla, Jeff Yan, and Peter Andras. 2012. Aimbot detection in online fps games using a heuristic method based on distribution comparison matrix. In *Neural Information Processing: 19th International Conference, ICONIP 2012, Doha, Qatar, November 12–15, 2012, Proceedings, Part V 19*. Springer, Springer Berlin Heidelberg, Berlin, Heidelberg, 654–661. https://doi.org/10.1007/978-3-642-34500-5_77
- [52] Su-Yang Yu, Nils Hammerla, Jeff Yan, and Peter Andras. 2012. A statistical aimbot detection method for online FPS games. In *The 2012 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8. <https://doi.org/10.1109/IJCNN.2012.6252489>

A Consensus algorithm

A.1 Parameter selection

The algorithm has been designed very carefully to consider both **error vote** and **rigged vote**. The discussion about **rigged vote** was already held in the paper, so we skip its explanation now. In the case of **error vote**, the history queue (Q_N) has been introduced to prevent decreasing in **Validity** score of normal users by **error vote**. To give weights on the user’s ballot and history, two variables of W_o and W_h have been set. In our design, W_o and W_h are set to 0.05 and 0.01 each. The values only affect the final scores, so we have to discuss the rate between the two values. W_h is set to one-fifth of W_o , and this setting performs ideally when the accuracy of the detection method is 80%. As an illustration, for normal users, it is reasonable to expect that they would have two **error vote** in the history queue on average, since the maximum size of the queue is ten ($10 \times \frac{(100-80)}{100}\%$). Therefore, the ratio of true and false ballots in the queue might be 8 : 2. In this case, even if they vote **error vote** once more, it will not affect their **Validity** scores. However, if someone starts to say lies over previous assumption, it will start to lose its influence on vote.

A.2 Customized algorithm

In addition to the consideration in the previous section, the details of the algorithm can be found on [Algorithm 2](#).

- ① Limit voters to participant in battlefield (c_cnt , n_cnt). When choosing the **Ground truth**, we don’t consider users who didn’t participate to *vote*.
- ② We don’t consider the **Self voting** into account.
- ③ The **Dubious** score is updated when the majority of users agree that the *target_user* is a cheating user.

Algorithm 2: Customized version**GT:** Ground truth**Users:** List of users W_v : Weight for Validity W_h : Weight for History Q_N : Queue for voting record of userN D_N : Dubious score of userN V_N : Validity score of userN ($0 < V_N < 1$) $Vote_{target}(N)$: A vote of userN for target**Function** score_evaluation(target_user):

①-Count users' votes and set ground truth

 $c_cnt \leftarrow 0$; $n_cnt \leftarrow 0$;**foreach** user in the Users **do** **if** user is target_user **then** continue; **if** $Vote_{target_user}(user)$ is True **then** $c_cnt \leftarrow c_cnt + 1$; **else if** $Vote_{target_user}(user)$ is False **then** $n_cnt \leftarrow n_cnt + 1$; **else** continue;

is Cheater

is Normal user

②-Skip self voting

Pass users who didn't vote

if $c_cnt > n_cnt$ **then** $GT \leftarrow \text{True}$;**else** $GT \leftarrow \text{False}$;**foreach** user in the Users **do** **if** user is target_user **then** continue; $t_cnt \leftarrow Q_{user}.\text{count}(\text{True})$; $f_cnt \leftarrow Q_{user}.\text{count}(\text{False})$; $hist_record \leftarrow t_cnt - f_cnt$; **if** $Q_N.\text{length}() > 10$ **then** $Q_N.\text{deque}()$; **if** $Vote_{target_user}(user) \neq GT$ **then** user_lied $\leftarrow \text{True}$; **else** user_lied $\leftarrow \text{False}$; **if** user_lied is True **then** $V_{user} \leftarrow V_{user} - W_v + hist_record * W_h$; $Q_{user}.\text{enqueue}(\text{False})$; **else** $V_{user} \leftarrow V_{user} + W_v + hist_record * W_h$; $Q_{user}.\text{enqueue}(\text{True})$;

②-Skip self voting

③-Calculate Dubious when majority agreed

if $c_cnt > n_cnt$ and $c_cnt + n_cnt > 2$ **then**

④-Restrict voters'

if $Vote_{target_user}(user)$ is True **then** $D_{target_user} \leftarrow D_{target_user} + \frac{c_cnt}{c_cnt + n_cnt} V_{user}$; **else** $D_{target_user} \leftarrow D_{target_user} - \frac{n_cnt}{c_cnt + n_cnt} V_{user}$;

⑤-Vote only for target_user

Function do_vote(target_user, GT):

score_evaluation(target_user, GT)

- ④ Participated users have the same influence, by dividing $c_cnt + n_cnt$, and majority voting have more power, by multiplying c_cnt and n_cnt .
- ⑤ A *vote* occurs only for a *target_user*.